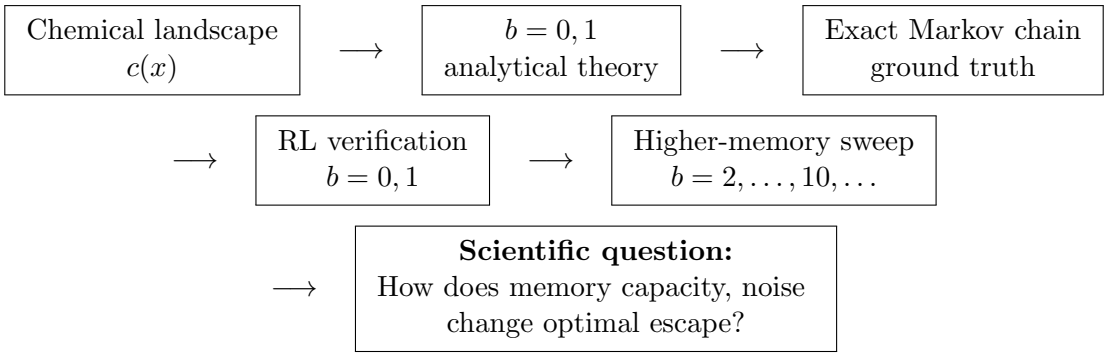
MAP: Chemotactic Escape from Information Traps

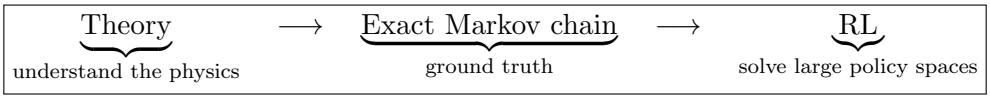
Two reference documents:

[**MAP**]	*this document*	`MAP_project.pdf`
[AD]	*Analytical Development*	`AD_analytical_development.pdf`
[CH]	*Exact Markov Chain Solver*	`CH_chain_solver.pdf`
[**RL**]	*RL document*	what to do once learning starts

1 The complete project strategy



The purpose of the analytical and exact calculations is *not* to replace reinforcement learning (RL), but to provide physical understanding, benchmark solutions, and a trustworthy ground truth before applying RL to larger memory spaces. The project can be organized as



1.1 Why start with $b = 0$ and $b = 1$?

Here b denotes the number of memory bits available to the agent. The cases $b = 0$ and $b = 1$ are particularly useful because they permit relatively simple analytical treatment, serve as **verification benchmarks**. We can compare

analytical theory $\approx$ Monte Carlo simulation $\approx$ exact Markov-chain calculation

before performing large RL sweeps. The project therefore has three computational levels:

$b = 0, 1$	$\longrightarrow$	closed-form analytical theory,
$b = 2, 3, \ldots$	$\longrightarrow$	exact Markov chain for fixed policies,
$b \gg 1$	$\longrightarrow$	RL for policy search.

The first two levels provide the ground truth needed to interpret the third.

2 The question, in one paragraph

An agent sits on the weaker of two chemical peaks. It must cross a valley to reach the better one. We ask: **how does the physics of that escape change when the agent is given one bit of memory?** Whether the *functional form* of the escape law changes.

The claim the project exists to establish.

$$\underbrace{\langle T_{\mathrm{esc}}\rangle \;\sim\; \frac{(x_{\min}-x_1)^2}{2D_{\mathrm{eff}}}}_{\text{0-bit: } \textit{diffusive}\text{, algebraic in geometry}} \qquad \longrightarrow \qquad \underbrace{\langle T_{\mathrm{esc}}\rangle \;\sim\; \tau_0\, e^{1.128\,\Delta U_{\mathrm{trap}}/\sigma}}_{\text{1-bit: } \textit{Arrhenius}\text{, exponential in barrier}}$$

A memoryless agent *cannot* build a confining drift (this is a theorem, not an observation), so it wanders out diffusively. A one-bit agent can, so it must climb over a barrier. The transition at $b_c = 1$ is a change of *kind*, not of degree.

3 The broad arc

	Question	Tool that answers it	Status
Q1	Given a policy, what does the physics do?	Analytical Development, then the Exact Markov Chain Solver	**Answered.** Closed forms derived and numerically confirmed.
Q2	What is the *best possible* policy at memory budget b?	optimise over policies, with the exact chain as the objective	**Open.** This is a computation, and **it needs no RL.**
Q3	Can an agent that is only given *rewards* discover that policy — and what does it learn?	reinforcement learning	**Open.** This is what RL is for.

Why the analytics being "already done" does not make RL pointless.

The analytics answer Q1 only, and only for the two policies simple enough to write down by hand (0-bit threshold, 1-bit persist-or-reverse). They say nothing about whether those analytical policies are any *good*.

Q2 asks for the optimum. That is a search over policies, and the exact chain makes the objective cheap and exact — so Q2 is best answered by direct optimisation, *not* by RL.

Q3 is the genuinely RL-shaped question: an agent given only a scalar reward, with no model of the landscape, must *find* the policy.

4 What is already settled, and what is not

Before writing code, be clear about which statements are finished and which are open. Do not re-derive the first group; do not assume the second.

Statement	Status	What you still do
0-bit agent cannot confine at a peak	**Proved** [AD]	Confirm numerically (it is a gate)
0-bit drift/diffusivity are functions of $c(x)$ alone	**Proved** [AD]	—
1-bit closed forms for μ and D_{eff}	**Derived**, checked vs. Monte Carlo to 4–5 decimals [AD]	Use as the target of a validation gate
Weak-signal escape slope $= 2/\sqrt{\pi} \approx 1.128$	**Derived** [AD]	*Measure* it; this is the headline test
Which stochastic-calculus convention the update realises (α)	**Open**	Measure it ([AD] §1.3); report the number
Correction factors for the single-sensor agent variant	**Measured** numerically [AD]	Choose a design (see §5)
Optimal policy at $b = 0, 1, 2$	**Open**	Compute it (Module 2) — no RL
Whether Q-learning finds that optimum	**Open**	This is the RL experiment
Whether memory is part of the reaction coordinate	**Open**; the instrument exists (Δq, [CH])	Compute per memory class

5 One design decision to make now

	Independent readings	Single sensor
How it compares	fresh gradient estimate each step	stores previous reading, differences it
Closed forms	**exact**	off by $\kappa_\mu = 0.72$, $\kappa_D = 0.52$
Headline slope	1.128	≈ 1.55
Exact chain exists?	**Yes** (2 memory states)	**No** (memory is continuous)

Use independent readings. It is the only version for which the analytics are exact *and* the exact chain exists — i.e. the only version where ground truth can be checked against. Build everything on it.

Run the single-sensor variant *once*, later, by Monte Carlo, as a deliberate comparison. The gap between them measures the cost of reusing a stale measurement, which is a small genuine result and nearly free once the pipeline works.

6 The build order

Six modules. Each depends only on the ones above it. **Do not skip ahead**; each row's "done when" is what licenses the next.

#	Module	What it does	Done when
0	`landscape.py`	$c(x)$, $c'(x)$, find $x_{\min}$, define ΔU_{trap}; three settings shallow→deep	plot + table of three ΔU_{trap} values
1	`chain.py`	the exact (x, M) Markov chain; stationary density, MFPT, committor, spectrum	row sums = 1; MFPT returns a finite number
2	`policies.py`	search for the *best* policy per memory class, using #1's exact MFPT as the objective	an optimal policy for $b = 0, 1, 2$
3	`sim.py`	vectorised Monte Carlo of the same model	**agrees with #1** to sampling error
4	`theory.py`	the continuum route: μ, D_{eff}, MFPT quadrature, saddle-point Kramers	the four-way table agrees (see gate)
5	`qlearn.py`	tabular Q-learning; the RL experiment	learned MFPT compared against #2's optimum

7 The gate that separates physics from learning

Gate

Compute $\langle T_{\text{esc}} \rangle$ at the same ΔU_{trap} by **four independent routes**:

(1) Monte Carlo (2) exact chain (3) MFPT quadrature (4) saddle-point Kramers

They form a ladder of increasing assumptions, so a disagreement tells you *which* assumption broke:

(1) vs (2)	Same model. A gap is a **bug** (or a boundary/initial-condition mismatch). Fix before anything else.
(2) vs (3)	The continuum approximation is failing — convention α, or action correlations, or $v\Delta t$ too large.
(3) vs (4)	The barrier is not deep enough for the asymptotic formula. **Expected**, tens of percent. Report it.

Do not start RL until (1) and (2) agree. Details in [CH] §1.6.

8 Where RL enters, and what it is for

Once the gate is passed we have: a validated simulator, an exact ground truth for any fixed policy, and the true optimum per memory class. *Now* RL has something to be measured against.

1. **Build it on the same state representation** already fixed in Module 1. Do not invent a new

discretisation at this point.

2. **Reward on the true field**, $r_t \propto c(x_t)$, not on the noisy reading. Partial observability belongs in the *state*, not the objective.

3. **Train with exploration on; measure with exploration off.** ε-greedy is training machinery; σ and D are physics. Never let them mix.

4. **The result is the gap.** Compare the learned policy's MFPT to Module 2's optimum for the *same* memory budget. "The agent fell X% short of the known optimum" is a real, quantitative finding. "The agent failed" would not have been.

5. **Then look at what it learned.** Read the memory-update rule off the trained table. This is the interpretability payoff, and it is the reason for using tabular Q-learning rather than a network.

If RL matches the optimum: you have shown the physics is learnable from reward alone, and you can push to $b = 2$ and beyond, where hand analytics stop and RL becomes the only tool.

If RL falls short: that is also a result, and a more interesting one — but only because you have an exact optimum to compare against. Without Module 2 it would be indistinguishable from a bug.

9 Sequence of works

Build	Produce
Module 0; read papers	landscape plot, three ΔU_{trap} values, the no-confinement proof
Module 1	exact MFPT for the 1-bit agent; stationary density
Module 3, then Module 4	Monte Carlo agrees with the chain; the $\ln\langle T_{\text{esc}}\rangle$ vs. $\Delta U_{\text{trap}}/\sigma$ plot with slope ≈ 1.128
Module 2; assemble the gate	optimal policy per memory class; the four-way table
Module 5; RL	

10 The short list of things

Each of these is explained in full in [AD] or [CH].

1. **Sweep ΔU_{trap} at fixed σ**, never σ at fixed ΔU_{trap}. The prefactor $\tau_0 \propto \sigma$, so the second sweep is not a straight line and the fitted slope is wrong. [AD]

2. **Match the grid to the step**, $h = v\Delta t/K$. Otherwise the chain acquires a large silent error at small D. [CH]

3. **Fit only the tail** of the first-passage histogram. The chain tells you the window: $\tau_{\mathrm{rel}} = -\Delta t / \ln|\lambda_2|$. [CH]

4. **Watch the parity mode.** At $D = 0$ the lattice produces a spurious eigenvalue near $-\lambda_1$ that will masquerade as non-exponential statistics. [CH]

5. **Measure the convention α; don't assume it.** And do not fit $\rho_s \propto D_{\mathrm{eff}}^{-\alpha}$ naively — that fit is only valid when the drift vanishes. [AD]

6. **Declare your initial condition.** Point start and quasi-stationary start give different numbers, both correct. Use the same one everywhere. [CH]

7. **Report distributions, not just means.** And report negative results — "no sharp transition" is a finding.

Computational Roadmap: From Known Physics to RL

Step 1: Construct the chemical landscape

$$c(x) = a_1 \exp\left[-\frac{(x-x_1)^2}{2s_1^2}\right] + a_2 \exp\left[-\frac{(x-x_2)^2}{2s_2^2}\right].$$

Find $x_1, x_{\min}, x_2$ and define the trap barrier

$$\boxed{\Delta U_{\text{trap}} = c(x_1) - c(x_{\min}).}$$

Check: plot $c(x)$ and mark $x_1, x_{\min}, x_2$.

Step 2: Establish numerical convergence

The stochastic dynamics are

$$x_{t+\Delta t} = x_t + v a_t \Delta t + \sqrt{2D\Delta t}\,\xi_t, \qquad \xi_t \sim \mathcal{N}(0,1).$$

Before studying physics, check numerical convergence.

Parameter	Suggested sweep	Check
Δt	0.1, 0.05, 0.02, 0.01, 0.005	T_{MFPT} converges
n	50, 100, 200, 400	spatial results converge
N_{MC}	10^3, 10^4, 10^5	statistical error decreases

$$\boxed{T_{\text{MFPT}}(\Delta t, n, N_{\text{MC}}) \longrightarrow \text{converged value.}}$$

Step 3: Validate the two simplest policies

The sensor measures

$$\tilde{c}_t = c(x_t) + \sigma\eta_t.$$

0-bit ($b = 0$):

$$a_t = \pi(\tilde{c}_t).$$

The action depends only on the instantaneous concentration.

$$\boxed{\text{Check: no drift asymmetry about the peak.}}$$

Two positions with equal concentration cannot be distinguished:

$$c(x_L) = c(x_R) \quad \Rightarrow \quad \mu(x_L) = \mu(x_R),$$

so a restoring drift on both sides of the peak is impossible.

1-bit ($b = 1$):

$$\Delta\tilde{c}_t = \tilde{c}_t - \tilde{c}_{t-1},$$

$$a_t = \begin{cases} a_{t-1}, & \Delta\tilde{c}_t > 0, \\ -a_{t-1}, & \Delta\tilde{c}_t < 0. \end{cases}$$

For the specified sensing convention, check

$$\boxed{\mu(x) = v\,\mathrm{sign}(c')\,\mathrm{erf}\left(\frac{|c'|v\Delta t}{2\sigma}\right)}$$

against simulation.

$$\boxed{\text{Simulation} \approx \text{analytical prediction.}}$$

Step 4: Calculate escape/MFPT

Define an absorbing boundary $x_b > x_{\min}$. For each trajectory,

$$T_i = \text{first time } x(t) \text{ reaches } x_b.$$

The Monte Carlo estimate is

$$\boxed{T_{\mathrm{MFPT}} = \frac{1}{N_{\mathrm{MC}}} \sum_{i=1}^{N_{\mathrm{MC}}} T_i.}$$

Also plot the complete first-passage distribution

$$P(T_{\mathrm{esc}}).$$

Check: vary the absorbing boundary slightly,

$$x_b = x_{\min} + \delta,$$

and verify that the physical conclusions are insensitive to δ.

Step 5: Main pre-RL physics sweep

Do **not** sweep every parameter simultaneously at first. First vary one physical parameter at a time.

Parameter	Suggested values	What it tests
a_1/a_2	0.5, 0.75, 1, 1.33, 2	trap asymmetry
$\Delta U_{\mathrm{trap}}/\sigma$	1, 2, 3, 4, 5, 6, 8	barrier strength
σ	0.05, 0.1, 0.2, 0.4	sensing noise
D	10^{-3}, 10^{-2}, 10^{-1}	Brownian diffusion
v	0.5, 1, 2	active motion

The most important sweep is

$$\boxed{\frac{\Delta U_{\mathrm{trap}}}{\sigma} \quad \longrightarrow \quad T_{\mathrm{MFPT}}.}$$

For $b = 1$, plot

$$\boxed{\ln T_{\mathrm{MFPT}} \quad \text{vs.} \quad \frac{\Delta U_{\mathrm{trap}}}{\sigma}.}$$

The weak-signal prediction is

$$\ln T_{\mathrm{MFPT}} \simeq \ln \tau_0 + \frac{2}{\sqrt{\pi}} \frac{\Delta U_{\mathrm{trap}}}{\sigma},$$

with

$$\boxed{\frac{2}{\sqrt{\pi}} \simeq 1.128.}$$

Check: fit the slope and compare with the theoretical value for the same sensing convention. If a different convention gives a different slope, identify the source rather than treating it as an RL effect.

Step 6: Exact Markov-chain benchmark

For a fixed policy π, discretize position into n bins and memory into 2^b states:

$$M \in \{0, \ldots, 2^b - 1\}.$$

The complete state is

$$\boxed{S = (x_{\mathrm{bin}}, M).}$$

Therefore,

$$\boxed{N_{\mathrm{states}} = n\, 2^b.}$$

Construct the transition matrix

$$P^{\pi}_{ij} = P(S_{t+1} = j \mid S_t = i, \pi).$$

For transient states,

$$\boxed{(I - P^{\pi}_{\mathrm{trans}})T = \mathbf{1}.}$$

This gives the MFPT of the discretized process without Monte Carlo sampling error.

$$\boxed{T_{\mathrm{Monte\,Carlo}} \;\approx\; T_{\mathrm{Markov}}}$$

for converged n. Use this exact benchmark for selected low-memory cases rather than attempting it indiscriminately at large b.

Step 7: RL validation

Only after Steps 1–6 pass, introduce RL.

State:

$$S_t = (x_{\text{bin}}, M_t),$$

actions:

$$a_t \in \{-1, +1\}.$$

Use Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right].$$

First train and test

$$\boxed{b = 0,\ 1.}$$

Require

$$\boxed{T_{\text{RL}} \approx T_{\text{MC}} \approx T_{\text{Markov}}.}$$

If this fails, **do not proceed to large** b; check the state representation, reward, exploration, timestep, boundary condition, and transition implementation.

Step 8: The actual RL research problem

After validation, increase memory:

$$\boxed{b = 0, 1, 2, 3, 4, 5, 6, 8, 10.}$$

Since

$$2^b = 1, 2, 4, 8, 16, 32, 64, 256, 1024,$$

the state space grows rapidly.

For each b:

$$\text{train RL} \rightarrow \text{freeze policy} \rightarrow \text{evaluate with independent trajectories} \rightarrow T^*_{\text{MFPT}}(b).$$

Finally plot

$$\boxed{T^*_{\text{MFPT}} \quad \text{vs.} \quad b.}$$

Final student workflow

Step	Student task	Must check
1	Build $c(x)$	$x_1, x_{\min}, x_2, \Delta U$
2	Convergence	$\Delta t, n, N_{\mathrm{MC}}$
3	$b = 0$	no confining drift
4	$b = 1$	analytical $\mu(x)$
5	MFPT	$P(T)$, T_{MFPT}
6	Barrier sweep	slope of $\ln T$ vs. $\Delta U/\sigma$
7	Markov chain	$T_{\mathrm{MC}} \approx T_{\mathrm{Markov}}$
8	RL: $b = 0, 1$	$T_{\mathrm{RL}} \approx$ benchmark
9	RL: high b	$T^*_{\mathrm{MFPT}}(b)$

The scientific questions are:

Does one bit provide most of the benefit?
Does performance improve with memory?
Does it saturate at a finite b?
Is there a critical memory scale?
How does the answer depend on $\Delta U/\sigma,\ D,\ v,\ a_1/a_2$?